DAGSmith: Dependency-Aware Rewriting for dbt-Style SQL Pipelines

Jie Liu
jiezzliu@umich.edu
Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Lin Ma
linmacse@umich.edu
Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Barzan Mozafari
mozafari@umich.edu
Computer Science and Engineering
University of Michigan, Ann Arbor
USA

ABSTRACT

Modern analytics is increasingly organized as recurring SQL pipelines rather than isolated SQL statements. Tools such as dbt let teams write each transformation as SQL and make dependencies between transformations explicit, producing directed acyclic graphs (DAGs) with hundreds or thousands of interdependent SQL models. This structure creates an optimization gap. Traditional query optimizers and source-to-source query rewriters operate one query at a time, while materialized-view selection and multi-query optimization address narrower forms of reuse. They do not directly exploit the pipeline-level information exposed by explicit dependencies: how intermediate results are consumed, which downstream outputs depend on each computation, where expensive work occurs relative to data reduction, which results should be persisted, and how refresh schedules relate to input changes and output demand.

We introduce DAGSmith, a dependency-aware source-to-source rewriting system for SQL pipeline DAGs. To the best of our knowledge, DAGSmith is the first holistic rewriting system for this setting. DAGSmith treats explicit dependencies as optimization signals. It analyzes compiled SQL together with dependency edges, uses an LLM to propose pipeline-level refactorings, separates proposal, criticism, SQL generation, and equivalence checking to reject unsafe rewrites, retunes persistence choices with a learned cost model, and selects a compatible set of rewrites globally. This enables dependency-edge simplification, non-local semantic reuse, downstream-aware pruning, pipeline-aware work placement, rewrite-materialization co-optimization, and frequency-aware optimization while keeping the pipeline executable by the same SQL engine and orchestration framework. Our evaluation shows that DAGSmith reduces end-to-end warehouse cost by **XX%** over the original pipeline, by **YY%** over materialization-only tuning, and by **ZZ%** over single-query rewriting baselines, while improving sink refresh latency by **LL%**.

1 INTRODUCTION

Rising SQL Complexity — SQL is no longer only a hand-written interface for individual analytical questions. It is increasingly the implementation language for recurring data products: governed dashboards, AI-ready feature tables, self-service BI datasets, and organization-wide reporting layers. At the same time, more SQL is produced by BI systems, code generators, and AI-assisted development tools rather than written from scratch. In fact, recent studies report that 70% of analytics professionals use AI for code development [9], and that BI-generated SQL contain extremely complex scalar expressions and relational operator trees [20]. This has created a major efficiency gap between the complexity of the SQL queries generated and what today's query optimizers are capable of effectively optimizing. Source-to-source query rewriting is the traditional way to narrow this gap: a rewriter transforms complex SQL into an equivalent form that the query optimizer is more likely to turn into an efficient plan. However, query rewriting has traditionally treated one SQL statement as the unit of optimization. Unfortunately, the effectiveness of this one-query-at-a-time approach is increasingly limited by a second trend: modern SQL is no longer produced and executed only as isolated statements, but as recurring transformation pipelines.

Rise of SQL Pipelines — Data teams are increasingly building large, recurring SQL pipelines. This growth is driven by complex business logic that must be computed repeatedly and reused consistently across governed dashboards, AI-ready feature tables, self-service BI datasets, compliance reports, and downstream applications. One of the most widely adopted tools for these workflows is *dbt* (the data build tool) [11], reported to be used by 50K+ data teams weekly across the world [7]. dbt lets teams write transformations as version-controlled SQL `SELECT` statements, declare dependencies among them, and compile the resulting graph into SQL jobs executed by a data warehouse. Figure 1 gives a toy example. A raw orders table feeds a cleaning transformation; the cleaned result feeds both a revenue table and a daily active-customer table. The SQL text explains each local transformation, while the edges explain how the transformations compose into a pipeline. This trend is much broader than dbt. SQLMesh [19], Dagster asset graphs [4], Airflow directed acyclic graphs [2], Prefect flows [15], Argo Workflows [3], Databricks Workflows [5], and medallion lakehouse pipelines [6] all represent recurring data transformations through explicit dependencies. In this paper, we target the SQL subset of this broader pattern, which we refer to as *SQL pipeline DAG*: a recurring directed acyclic graph whose nodes are SQL transformations or SQL-produced tables/views, and whose edges record that one node reads another node's output.

Pipeline Optimization Gap — These pipeline DAGs often include hundreds or thousands of SQL transformations that reference one another. Their scale makes them difficult to optimize with techniques designed for a single SQL statement or a flat workload: inlining every referenced transformation into each downstream query would produce deeply nested SQL with tens or hundreds of layers whose size and complexity are well beyond what today's optimizers can even tackle. In particular, existing *Query rewriting* techniques, whether traditional rule-based systems [21] or modern synthesis- or LLM-based techniques [10, 13, 18], rewrite one SQL statement at a time. This scope is useful for improving a standalone query,

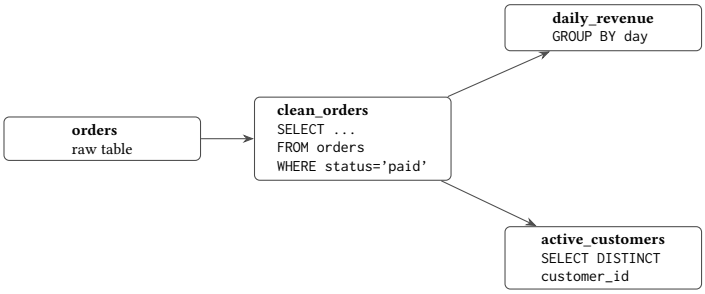


Figure 1: A toy SQL pipeline DAG. Nodes are SQL transformations or SQL-produced tables; edges show which transformation consumes another transformation's output.

but it is too narrow for optimizations whose correctness or benefit depends on the surrounding pipeline, such as removing work no final output uses, reusing equivalent logic across separate transformations, or changing refresh frequency based on input changes and output consumption. *Multi-query optimization* techniques [16, 17] consider multiple queries together, but their classical goal is to share common scans, joins, or intermediate results across a batch of submitted queries. They do not generally rewrite a SQL pipeline by adding, removing, splitting, or merging transformations based on downstream demand and refresh behavior. **Materialized-view selection** [1, 12] chooses intermediate query results to precompute and rewrites future queries to read those precomputed results. This improves reuse when the relevant computation has already been identified, but it does not by itself determine that a pipeline should drop unused work, factor repeated logic written in different forms, move expensive operations after data reduction, or refresh different parts of the pipeline at different rates. **Workflow schedulers** [2–4, 15] know dependencies, but usually optimize execution order, failure recovery, retries, or resource allocation rather than the SQL semantics of the transformations themselves.

Dependencies as Signals — Our key insight is that explicit dependencies are not merely execution constraints; they are signals that can be leveraged for optimization. In particular, a SQL pipeline DAG exposes six kinds of information that are hidden or much harder to recover when queries are optimized independently: **(1) Edge semantics:** producer-consumer edges reveal how one SQL transformation's output is used by later transformations; **(2) Cross-graph redundancy:** similar business logic can appear in distant parts of the graph, even when SQL text differs; **(3) Downstream demand:** downstream dependencies reveal which columns, joins, filters, and intermediate results can affect final outputs; **(4) Operator placement:** the graph reveals where expensive operations occur relative to data-reducing filters, projections, joins, and aggregations; **(5) Rewrite-persistence coupling:** reuse patterns and fanout reveal when a rewritten model should be materialized as a stored result, and when it should instead be inlined or recomputed by downstream transformations; and **(6) Refresh patterns:** schedules and source update rates reveal how often inputs change and outputs are consumed.

Our Approach — In this paper, we introduce DAGSmith, the first (to the best of our knowledge) holistic, dependency-aware source-to-source rewriting system for SQL pipeline DAGs. DAGSmith exploits the exposed dependency structure to perform six classes of optimization: **(1) dependency-edge simplification:** remove or simplify dependency edges whose purpose can be expressed more directly, while preserving final outputs; **(2) non-local semantic reuse:** factor repeated work into a shared computation across non-adjacent transformations; **(3) downstream-aware pruning:** remove work that is provably irrelevant to every final output; **(4) pipeline-aware work placement:** move expensive work after data-reducing steps as long as final outputs are not impacted; **(5) rewrite-materialization co-optimization:** decide when rewrite benefits are outweighed by materialization cost, and when materializing a rewritten result makes the rewrite worthwhile; and **(6) frequency-aware optimization:** align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers.

DAGSmith first analyzes compiled SQL and dependency edges to identify regions likely to contain these opportunities. It then asks an LLM to reason over the relevant subgraph and propose concrete refactorings, but separates proposal, criticism, SQL generation, and equivalence checking so unsafe or counterproductive rewrites can be rejected before adoption. Adopted candidates are not evaluated in isolation: DAGSmith retunes persistence choices for each candidate using a learned cost model and an iterative local linearization procedure, then solves a global selection problem to choose a non-conflicting subset of rewrites. Finally, frequency information from source update rates and output demand reweights cost estimates, detects over-scheduled work, and generates splitting candidates when slow-changing logic is embedded in fast-refreshing transformations. The result is a rewritten pipeline that remains executable by the same SQL engine and orchestration framework, but is significantly more performant.

Contributions — This paper makes the following contributions:

(1) We formulate dependency-aware source-to-source rewriting for SQL pipeline DAGs. We present the first holistic approach to this problem (to the best of our knowledge) by identifying the pipeline-level signals exposed by explicit dependencies and the rewriting opportunities they enable: dependency-edge simplification, non-local semantic reuse, downstream-aware pruning, pipeline-aware work placement, rewrite-materialization co-optimization, and frequency-aware optimization.

(2) We present the design of DAGSmith, which combines structural graph analysis, LLM-guided refactoring, separated proposal, criticism, SQL generation, and equivalence checking, learned cost modeling, persistence tuning, frequency-aware scheduling, and global candidate selection.

(3) We conduct a comprehensive evaluation, showing that DAGSmith reduces the end-to-end completion time and monetary cost by **XX%** and **WW%** over the original pipeline, by **YY%** and **WW%** over materialization-only tuning, and by **ZZ%** and **WW%** over single-query rewriting baselines, respectively.

The rest of the paper is organized as follows. Section 2 defines the setting and illustrates the opportunity space. Section 3 presents

Table 1: Optimization signals exposed by explicit SQL pipeline dependencies.

#	What explicit dependencies expose	How DAGSmith leverages them for optimization
1	Producer-consumer edges reveal how one SQL transformation's output is used by later transformations.	**Dependency-edge simplification:** remove or simplify dependency edges whose purpose can be expressed more directly, while preserving final outputs.
2	Similar business logic can appear in distant parts of the graph, even when SQL text differs.	**Non-local semantic reuse:** factor repeated work into a shared computation across non-adjacent transformations.
3	Downstream dependencies reveal which columns, joins, filters, and intermediate results can affect final outputs.	**Downstream-aware pruning:** remove work that is provably irrelevant to every final output.
4	The graph reveals where expensive operations occur relative to data-reducing filters, projections, joins, and aggregations.	**Pipeline-aware work placement:** move expensive work after data-reducing steps as long as final outputs are not impacted.
5	Reuse patterns and fanout reveal when a rewritten model should be materialized as a stored result, and when it should instead be inlined or recomputed by downstream transformations.	**Rewrite-materialization co-optimization:** decide when rewrite benefits are outweighed by materialization cost, and when materializing a rewritten result makes the rewrite worthwhile.
6	Schedules and source update rates reveal how often inputs change and outputs are consumed.	**Frequency-aware optimization:** align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers.

the design of DAGSmith. Section 4 outlines the evaluation plan, Section 5 discusses prior work, and Section 6 concludes.

2 OPPORTUNITIES IN SQL PIPELINE DAGS

Before presenting our approach, we define the optimization setting that DAGSmith targets and illustrate, through three examples drawn from or inspired by real-world analytics pipelines, the kinds of dependency-aware optimizations that this setting enables but that fall outside the reach of existing techniques.

2.1 Setting

A *SQL pipeline DAG* is a directed acyclic graph $G = (V, E)$ whose nodes are SQL transformations or SQL-produced tables/views, and whose edges record that one node reads another node's output. We use "DAG" in the standard sense: the dependency graph is directed, and cycles are disallowed so the transformations can be evaluated in dependency order. A node may be implemented as a dbt *model* (dbt's term for a SQL transformation that produces a table or view), a notebook cell that issues SQL, a warehouse task, or a table produced by an orchestration framework. The graph may be declared directly, as in Airflow [2] and Argo [3], inferred from dataflow, as in Prefect [15], or induced by model references, as in dbt [8] and SQLMesh [19]. In asset-oriented systems such as Dagster [4], the same idea appears as an asset graph: nodes are persistent data assets and edges describe how one asset is derived from another.

dbt is the running example in this paper because its SQL transformation boundaries and dependency declarations make the setting especially explicit. However, the optimization problem is broader. Databricks Workflows [5] and Lakeflow Jobs [5] expose task dependencies as a job DAG; Databricks' medallion architecture [6] encourages multi-hop transformations from bronze to silver to gold tables; Luigi [14] and the systems above all represent recurring data work through dependency graphs. DAGSmith targets the subset of these pipelines where the node logic is available as SQL, or can be compiled to SQL, so that the optimizer can reason about relational semantics rather than only task ordering.

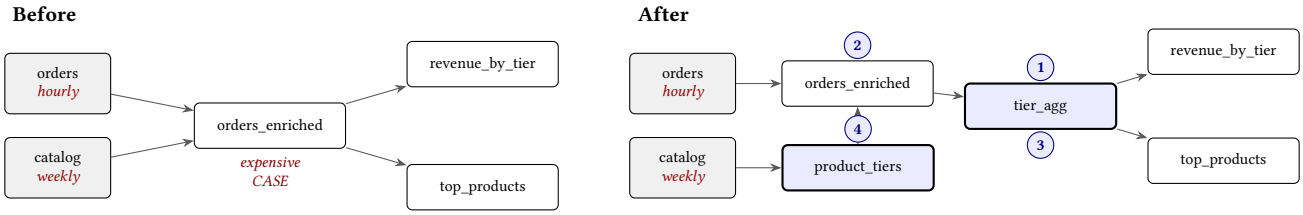
This setting differs from traditional query optimization in six ways that matter for our problem. Figure 2 previews several of the changes these differences make possible.

(1) Cross-boundary rewriting. The graph identifies explicit boundaries between SQL transformations: which output is produced at one node, which later nodes read it, and which final outputs must remain unchanged. These boundaries are optimization choices, not just execution constraints. A dependency-aware optimizer can therefore bypass an intermediate node, inline part of a producer, pull a predicate from a consumer into a producer, or replace a dependency on one node with an equivalent expression over another node, as long as the final pipeline outputs are preserved. This is stronger than expanding a view inside one query: the optimizer can change how SQL is divided across transformations, not merely optimize the compiled text of one statement.

(2) Non-local semantic reuse. The graph exposes similar business logic that appears in different parts of the pipeline, often under different names or SQL structure. A dependency-aware optimizer can factor repeated work into a shared computation and rewrite consumers to use that shared result. This is not limited to syntactic common subexpressions: two transformations may compute the same logical quantity using different predicates, aliases, joins, or intermediate names.

(3) Downstream-aware pruning. Downstream transformations reveal which columns, joins, or intermediate results are never consumed by any final output. A dependency-aware optimizer can remove such work safely because the graph proves that no later transformation depends on it. A one-query rewriter can remove locally dead expressions inside one statement, but it cannot know whether an intermediate result produced for the pipeline is actually read later.

(4) Pipeline-aware work placement. The graph shows where expensive operations occur relative to filters, projections, aggregations, and joins. A dependency-aware optimizer can move expensive work after data-reducing steps as long as final outputs are not impacted. This opportunity is pipeline-level because the



(1) *Cross-boundary rewriting.* Producer-consumer edges identify SQL boundaries that can be replaced, bypassed, or shifted when an equivalent relationship exists across the boundary.
(2) *Non-local semantic reuse.* Similar logic in separate parts of the graph can be factored into a shared computation even when the SQL text differs.
(3) *Rewrite-materialization co-optimization.* A shared intermediate is useful only under the right persistence choice; rewriting and persistence must be optimized together.
(4) *Frequency-aware optimization.* The expensive classification depends only on weekly data but is recomputed inside an hourly transformation; splitting it onto the slow cadence requires a pipeline rewrite invisible to per-query cost.

Figure 2: A compact example of optimization opportunities exposed by an explicit SQL pipeline DAG. Once dependencies are visible, the optimizer can rewrite across transformation boundaries, factor repeated logic, co-optimize rewriting with persistence, and split slow-changing logic off fast refresh paths.

data-reducing step and the expensive operation may be separated across transformation boundaries.

(5) Rewrite-materialization co-optimization. The graph shows which rewritten results would be reused and where recomputation would occur if they are not persisted. A dependency-aware optimizer can choose rewrites and persistence choices together, since a rewrite can create new persistence opportunities and persistence can determine whether a rewrite helps. For example, a shared transformation may reduce cost when written once as a table but increase cost when inlined as a view under several consumers.

(6) Frequency-aware optimization. Schedules and source update rates reveal how often inputs change and outputs are consumed. A dependency-aware optimizer can align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers. In cloud warehouses, these rates translate directly into cost and latency objectives: if a weekly source feeds an hourly transformation, any computation that depends only on the weekly source is being repeated unnecessarily; if an output is consumed daily, hourly refresh may be waste even when the SQL statement itself is individually optimized.

Together, these properties define a distinct optimization setting. The optimizer is reasoning not about one query, nor merely about an unordered set of independent queries, but about a recurring SQL pipeline program whose graph structure, final outputs, persistence choices, and refresh behavior interact.

2.2 Opportunity Taxonomy

The properties above create a broader opportunity space than the three examples in this section can cover exhaustively. We organize the space into six recurring classes that map directly to the components of DAGSmith in Section 3.

Cross-boundary rewriting. Producer-consumer edges expose transformation boundaries that may be bypassed, shifted, or replaced. The optimization opportunity is to replace a cross-node dependency with a cheaper equivalent relationship while preserving the final outputs of the pipeline.

Non-local semantic reuse. Similar logic can appear in separate parts of the pipeline, often under different names or SQL structure. The opportunity is to factor repeated work into a shared computation and rewrite consumers to use that shared result.

Downstream-aware pruning. Later transformations reveal which columns, joins, or intermediate results are never used. The opportunity is to remove work that is provably irrelevant to all downstream outputs.

Pipeline-aware work placement. The graph shows where expensive operations occur relative to filters, projections, aggregations, and joins. The opportunity is to move expensive work after data-reducing steps as long as final outputs are not impacted.

Rewrite-materialization coupling. The best logical structure depends on which nodes are persisted. A rewrite that creates a shared node only helps if that node is persisted appropriately; inlining it as a view may simply duplicate the work downstream.

Frequency-aware optimization. The cost of a transformation is not just its per-run cost. It is the cost multiplied by how often the transformation must produce useful new output. The optimizer can align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers.

Global rewrite composition. The six opportunities above can interact. Two refactorings may touch the same transformation, make incompatible assumptions about an intermediate result, or interact through persistence choices. A dependency-aware optimizer needs a global selection step rather than a greedy sequence of local rewrites.

We now illustrate three representative classes that DAGSmith discovers. Each requires the optimizer to read and reason about SQL definitions across transformation boundaries within the DAG. The

SQL in each example is simplified for presentation; domain-specific details are abstracted away.

Example 1: Non-local semantic reuse. A pipeline contains seven classifier transformations that each tag orders with a service category (`dialysis`, `psychiatric`, and five others). The input is a `claims` table in which each row represents one item of an order, so a single order has many rows. Each classifier scans `claims`, keeps rows whose codes match its category's code list, and returns the distinct `order_id`s of those rows. The seven classifiers have *different* SQL: different predicates, different code vocabularies, and in some cases different join structures. Two representative classifiers are shown on the left of Figure 3.

Before: 7 classifiers (2 shown)

```sql
-- dialysis_classifier
SELECT DISTINCT order_id
FROM {{ ref('claims') }}
WHERE bill_code LIKE '72%'
   OR taxonomy IN (set_dia_tax)
   OR diagnosis IN (set_dia_dx)


-- psychiatric_classifier
SELECT DISTINCT order_id
FROM {{ ref('claims') }}
WHERE revenue IN (set_psy_rev)
   OR diagnosis BETWEEN
        '290' AND '319'
```

After: precomputation+lookups

```sql
-- order_flags (new)
SELECT order_id,
  MAX(CASE WHEN bill_code LIKE '72%'
    OR taxonomy IN (set_dia_tax)
    OR diagnosis IN (set_dia_dx)
    THEN 1 ELSE 0 END) AS f_dia,
  MAX(CASE WHEN ...
    THEN 1 ELSE 0 END) AS f_psy,
  ...  -- 7 categories
FROM {{ ref('claims') }}
GROUP BY order_id

-- dialysis_classifier (rewritten)
SELECT order_id
FROM {{ ref('order_flags') }}
WHERE f_dia = 1
```

Figure 3: Non-local semantic reuse. Seven classifiers with no syntactic overlap perform the same semantic operation on a shared parent. DAGSmith synthesizes a new shared transformation that pre-aggregates all classifier flags in one pass.

The seven classifiers share no common subexpression, yet they all perform the same *semantic operation*: for each order, decide whether any of its rows match a category-specific predicate. DAGSmith recognizes this cross-transformation pattern and factors it. It synthesizes a new intermediate transformation `order_flags` that, in a single pass over `claims`, computes one boolean flag per category by aggregating each category's predicate with `MAX(CASE WHEN ... THEN 1 ELSE 0 END)` grouped by `order_id`. Each original classifier is then rewritten as a lightweight lookup: select the orders whose corresponding flag in `order_flags` is 1. Seven full scans and seven independent grouping passes over `claims` are replaced with one pre-aggregation pass and seven inexpensive boolean filters.

This optimization requires three capabilities that go beyond traditional techniques. First, the optimizer must recognize that seven syntactically dissimilar transformations perform the same semantic operation, which syntactic subexpression matching, the basis of multi-query optimization (MQO) [17] and common subexpression elimination, cannot do. Second, it must *invent* a new intermediate transformation absent from the original project; materialized view recommendation tools [1] synthesize candidate views, but they too rely on syntactic subexpression mining, of which the seven classifiers share none. Third, it must rewrite the dependent transformations to consume the new intermediate while preserving end-to-end equivalence, a multi-transformation restructuring that single-query rewrite systems [18, 21] do not attempt.

Example 2: Cross-boundary rewriting. The same project contains a classifier `nursing` that filters for skilled-nursing claims by facility code, then performs an anti-join against a sibling transformation `equipment` to exclude equipment-related claims (Figure 4, left). The anti-join forces the engine to produce `equipment`, build a hash table, and probe it for every row of `claims`; it also introduces a dependency edge that serializes the two transformations.

Before: anti-join

```sql
-- nursing
SELECT *
FROM {{ ref('claims') }} c
WHERE c.facility_code
        IN ('31','32')
  AND NOT EXISTS (
    SELECT 1
    FROM {{ ref('equipment') }} e
    WHERE e.order_id = c.order_id
      AND e.line_id = c.line_id)

-- equipment (just a filter)
SELECT *
FROM {{ ref('claims') }}
WHERE category = 'equipment'
```

After: predicate

```sql
-- nursing (rewritten)
SELECT *
FROM {{ ref('claims') }} c
WHERE c.facility_code
        IN ('31','32')
  AND (c.category IS NULL
    OR c.category <> 'equipment')



-- equipment is removed if no
-- other transformation reads it;
-- otherwise its definition
-- is unchanged.
```

Figure 4: Cross-boundary rewriting. By reading the SQL of the sibling transformation `equipment`, DAGSmith determines that the anti-join is equivalent to a scalar predicate on a shared parent. The hash anti-join, the auxiliary scan, and the dependency edge are all eliminated.

By reading the SQL definition of `equipment`, DAGSmith discovers that it computes nothing more than a single-predicate filter on a shared parent. The anti-join is therefore equivalent to an inline scalar predicate on `claims`, as shown on the right of the figure: the hash anti-join, the auxiliary scan, and the cross-transformation dependency are all eliminated. The same reasoning generalizes to every sibling classifier that uses `equipment` as an exclusion list, compounding the benefit across the pipeline.

The equivalence enabling this rewrite is not declared anywhere; it is *embedded in the SQL of another transformation file*. A traditional engine sees `equipment` either as a base table to be probed or as a view to be expanded into the local query, neither of which performs the cross-transformation analysis needed to eliminate the join entirely. We acknowledge a tradeoff: the rewrite couples `nursing` to the current definition of `equipment`, so a future change to that definition would require updating the rewritten consumers. In practice, stable upstream transformations such as `equipment` change rarely, and DAGSmith re-evaluates the project after each developer change, so the cost of re-deriving the rewrite is amortized over many executions.

Example 3: Frequency-aware optimization. Different sources in a SQL pipeline DAG update at different rates, and the structure of the DAG determines how much computation must be repeated at each high-frequency refresh. Consider a transformation that joins fast-changing transactional data with slow-changing reference data and applies expensive classification logic to the slow-changing side (Figure 5, left).

Before: hourly classification

```sql
-- order_summary (hourly)
SELECT o.order_id, o.amount,
  CASE
    WHEN p.code IN (set_A) THEN 'A'
    WHEN p.code IN (set_B) THEN 'B'
    ...
  END AS category
FROM {{ ref('orders') }} o
JOIN {{ ref('catalog') }} p
  ON o.product_id = p.product_id


-- orders: hourly
-- catalog: weekly
```

After: split by frequency

```sql
-- product_categories (weekly)
SELECT product_id,
  CASE
    WHEN code IN (set_A) THEN 'A'
    WHEN code IN (set_B) THEN 'B'
    ...
  END AS category
FROM {{ ref('catalog') }}

-- order_summary (hourly)
SELECT o.order_id, o.amount,
       pc.category
FROM {{ ref('orders') }} o
JOIN {{ ref('product_categories') }} pc
  USING (product_id)
```

Figure 5: Frequency-aware optimization. Splitting the transformation along the frequency boundary moves expensive classification onto a weekly cadence; the hourly transformation becomes a lightweight join.

Because `orders` changes hourly, `order_summary` re-executes hourly, and the multi-branch classification over `catalog` is recomputed on every hourly refresh, despite its inputs changing only weekly. DAGSmith traces the update frequency from each source through the DAG, identifies that this transformation has *mixed-frequency inputs*, and splits it along the frequency boundary. The expensive classification is extracted into a new slow-frequency transformation, and the original transformation becomes a lightweight equi-join. When this pattern recurs across many consumers of the same slow-changing reference data, the savings compound: each hourly refresh avoids redundant recomputation of logic whose inputs have not changed.

This optimization is invisible to optimizers that treat cost as a fixed per-query quantity: the benefit of splitting the transformation is realized only when the two halves are known to execute at different rates. Materialized view selection considers update cost but does not restructure the DAG to align computation with update frequency, and traditional query rewriters have no notion of refresh schedule at all.

2.3 Why Existing Techniques Fall Short

The three examples above are not isolated curiosities; they illustrate a general capability gap. Single-query rewrite systems [18, 21] operate within the boundary of one query and cannot restructure across transformation files. Multi-query optimization and common subexpression elimination [17] share intermediate computations across queries but rely on syntactic matching, so they miss the cross-transformation semantic equivalence in Example 1. Materialized view selection [1] chooses among existing view candidates and treats update cost as a static parameter rather than as a structural driver of DAG design (Example 3). Code clone detection and lineage tools can identify structural similarity or trace data provenance, but they do not perform the optimization itself.

The gap is not that any one of these techniques is poorly designed; it is that none of them targets the setting we are concerned with. Optimizing a SQL pipeline DAG requires an optimizer that can combine the six capabilities above: use producer and consumer context, find semantic reuse beyond syntactic overlap, prune work that no downstream output consumes, move expensive work after data-reducing steps, evaluate rewrites together with persistence choices, and account for recurring refresh behavior. The remainder of this paper presents an approach that integrates these capabilities into a coherent optimization framework.

3 APPROACH

lin: Okay in general, I think this section has way too many implementation details and less intuitive lessons. The purpose of a paper is not to document every details of the implementation so that others will recreate every single step. You'll have your code released and they can reference your code anyway. Rather, the purpose is do convery what are the key technical challenges to solve this problem, (intuitively) what are our solutions to solve these challenges and why we choose these solutions, and what lessons did we learn. Implementaiton details are helpful only if we have the space and if that supports our argument for the solution. But having too many can be distracting. I also think having 5 subsections (components) are a bit too many. I would suggest having only 3-4 subsections. (I think earlier you wrote there are 3 optimization dimensions? That might be a good breakdown for the subsections. It seems that in the ablation study you only categorize three optimizations anyway.) And within each subsection, you start with what technical challenges you're going to solve (and why such challenges exist in our problem), and intuitively how your solution works (and why it solves the challenges). A figure illustrating the sollution would be a plus. Then, you go into a little bit of details to just have the audience have a slighly deeper understanding. You can take a look at other papers (or even your previous papers) to see how long this should be. Readers/Reviewers won't appreciate if you add too many details and they couldn't easily get the big picture.

DAGSmith optimizes a SQL pipeline DAG along three interacting dimensions: graph refactoring, materialization, and refresh frequency. lin: I know Barzan is still reorganizing sections 0-2, but since we have three dimensions here, we know just condense the earlier six opportunites to three, and we have an optimization for each of the opportunity? We can probably connect each of the example in Section 2 to one dimension here as well. Wouldn't this be much more succinct and saving lots of space? The benefit of a refactoring depends on which nodes will be materialized, the optimal materialization configuration depends on the structure of the DAG, and both depend on how often each node executes. A pipeline with hundreds or thousands of models admits an enormous space of candidate refactorings, materialization assignments, and refresh schedules; exhaustive search is infeasible, and the three dimensions cannot be optimized in independent passes without missing interactions or producing decisions that conflict at the boundaries.

lin: I think we need to reference Figure 5 at the beginning of this paragraph DAGSmith addresses this challenge with a three-stage pipeline. The first stage *identifies* promising refactoring opportunities. Structural analyses of the pipeline's SQL rank model groups by a fingerprint-based score that estimates the potential benefit of joint refactoring; the top-ranked groups are then passed to an LLM, which proposes one or more concrete refactorings for each. lin: Since we mentioned LLM, I think we need to add one sentence to talk about the correctness guarantee. Otherwise, I think that would

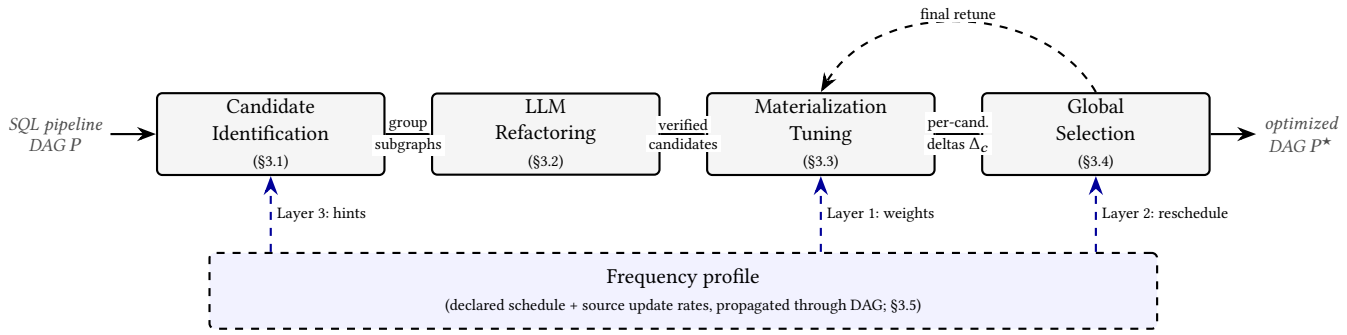


Figure 6: End-to-end pipeline of DAGSmith. The top row shows the four main stages and the artifacts that flow between them; after global selection, a final materialization pass retunes the combined DAG. The dashed band underneath is the frequency-aware overlay (§3.5): a single frequency profile feeds three of the four stages, contributing structural splitting hints, cost weights for materialization tuning, and schedule corrections at selection.

be the first question that the reviewer raises.Together, the structural analyses and the LLM constitute the candidate generator: the structural analyses decide *where* in the DAG to look, and the LLM decides *what* refactoring to perform. The second stage *evaluates* each candidate refactoring under its own optimal materialization configuration. A learned cost model predicts slot time for any DAG configuration, and an iterative local linearization procedure uses these predictions to find the materialization assignment that minimizes total cost for the candidate. The third stage *composes* the surviving candidates into a single optimized DAG. An ILP selects the best non-conflicting subset of refactorings, after which a final materialization pass tunes the combined DAG. The frequency dimension contributes to the pipeline at three levels: it weights the cost-model-driven decisions during materialization tuning and refactoring selection, it corrects over-scheduled refresh rates so that nodes run only as often as their inputs change, and it contributes a class of structural refactorings in the first stage, where the LLM identifies nodes that can be split along frequency boundaries.

The remainder of this section presents the components of DAGSmith in the order they appear in the pipeline. Section 3.1 describes the structural analyses that identify candidate model groups. Section 3.2 describes how the LLM produces concrete refactorings from the resulting subgraphs. Section 3.3 describes the cost model and the iterative local linearization procedure that finds the best materialization configuration for any given DAG structure. Section 3.4 presents the ILP that selects the best non-conflicting combination of refactorings. Section 3.5 extends the pipeline with frequency-aware weighting at the materialization and selection stages and introduces the frequency-driven splitting refactoring. lin: Not sure if this paragraph is necessary. Seems very long to just overview the structure of this section...

3.1 Candidate Identification

The first stage of the pipeline identifies regions of the DAG where a refactoring is likely to be worthwhile. A pipeline with hundreds of SQL nodes admits an enormous number of conceivable refactorings, so applying an LLM to every node group is prohibitively expensive and produces a high rate of trivial proposals. lin: I don't think we have established why we want to use LLM for refactoring yet. So I think either you have to establish that here, or just talk about refactoring is costly in general, without specifically emphasizing LLMs. DAGSmith addresses this with a division of labor: structural analyses over the pipeline's SQL surface candidate regions whose dataflow exhibits patterns associated with refactoring opportunities, and the LLM examines each region and decides what refactoring, if any, is appropriate. Structural analysis is cheap and is applied to every node in the DAG; LLM reasoning is expensive and is reserved for regions where structural evidence suggests it is likely to pay off.

lin: Before getting into all the details below, I think it would be good to have one paragraph to discuss intuitively how your analysis works and what it achieves (if you can pair that with a figure, that's even better). The rest of this subsection is just implementation details. If intuitively the method makes sense, we would be good. We can either keep the details below or just condense them if we're short on space.

Dataflow representation. Each node's SQL is compiled into a per-node dataflow representation that captures column derivations, joins, and aggregations as a tree of typed operators, with column references resolved across dependency boundaries. In dbt-style projects, these boundaries are expressed through the `ref()` graph. Consider three models:

```
A: SELECT k1, k2, SUM(c1) AS a1 FROM {{ ref('s1') }} GROUP BY k1, k2
B: SELECT k1, SUM(a1) AS b1 FROM {{ ref('A') }} GROUP BY k1
```

Model C performs both aggregations within a single SQL statement. At the SQL level, B's reference to A is opaque: the column `a1` is just a name produced by some upstream model. The dataflow representation resolves this reference, links the column in B back to its definition in A, and reconstructs the chain:

```
s1.c1 -> SUM by (k1, k2) -> SUM by k1 -> B's output
```

The same chain is contained inside C. The structural correspondence between the two-model composition and the single model is therefore visible in the representation, even though it is hidden by the `ref()` boundary in the source SQL. The analyses described

below operate on this representation, which is what allows them to recognize cross-model patterns that are invisible at the SQL level.

Reuse analysis. Reuse analysis identifies groups of models that perform the same expensive operation over related inputs. From each model's dataflow representation, it extracts a structural fingerprint summarizing the model's expensive operations, joins and group-by aggregations, together with their anchor columns (join keys and grouping keys) and input relation lineage. Models whose fingerprints share substantial structure are grouped into candidate sets. Each candidate set is scored by an estimate of joint refactoring potential, which weights the shared operation by its expected cost and by the number of models that share it.

A high reuse score is a *targeting signal*, not an instruction to materialize a shared subexpression. Reuse analysis subsumes the syntactic shared-subexpression case targeted by multi-query optimization and materialized view recommendation, but extends it to semantically equivalent operations that are syntactically dissimilar.

Prune analysis. Prune analysis targets joins, in two forms. The first is *redundant joins*: joins whose results are not consumed by any downstream model. These can be eliminated entirely once column lineage from leaf models confirms that no downstream consumer reads from the joined output. Semi-joins fall into this category when the filtering set is statically derivable from columns already present in the probed relation, in which case the join itself can be replaced with a predicate. The second form is *mistimed joins*: joins that are correct in semantics but are computed too soon. When one side of a join feeds into expensive downstream operations whose results do not depend on the other side's columns, the join inflates the input to those operations without contributing to their result. Pushing the join later, after the expensive operations have reduced the data, avoids carrying the join product through them. As an additional pattern, prune analysis also flags unused projected columns whose elimination simplifies upstream definitions.

What distinguishes prune analysis from local dead-code elimination in a single-query optimizer is its scope: a redundant or mistimed join in one model is invisible to a single-query rewriter, which has no view of how downstream models will use the result. Each detected candidate is scored by the cost of the affected operation, weighted by the number of downstream models whose simplified definitions benefit from the change.

From candidates to subgraphs. The two analyses produce a ranked stream of candidate model groups. From each group, a subgraph of the DAG is extracted that includes the candidate models together with a configurable neighborhood of upstream and downstream models, so that the LLM has enough surrounding context to reason about correctness. Subgraph size is bounded to keep the LLM input manageable; groups whose neighborhoods exceed the bound are skipped. The next subsection describes how the LLM consumes these subgraphs and produces concrete refactorings.

3.2 LLM Refactoring

lin: Well, why do we want to use an LLM to do this in the first place? We either have to establish this earlier or justify it here.

Asking an LLM to rewrite SQL that will be deployed against a real warehouse raises three problems. The proposed rewrite may run *slower* than the original, may be *incorrect*, or may be *too expensive to produce naively* when a candidate region contains many models. DAGSmith addresses these with a four-stage sequence: subgraph analysis and opportunity proposal, proposal filtering, refactoring, and equivalence verification. A separate, lighter mechanism handles batches of near-duplicate models and is described at the end of the section.

lin: Similarly, I think one paragraph to discuss intuitively how your approach works and what it achieves (better with a figure) can be valuable. The details can be reduce.

Subgraph analysis and opportunity proposal. The first LLM call ingests the subgraph's SQL and dependency graph and produces a domain-level reading of the region: a short semantic summary of each model and an account of the end-to-end business question the subgraph answers. This domain-level read is what allows subsequent stages to recognize semantically equivalent computations expressed in syntactically dissimilar SQL, the capability that distinguishes DAGSmith from purely syntactic rewriters. The headline output is a list of *optimization opportunities*: each names the CPU-bound computation it would eliminate or shrink and the mechanism by which it would do so. Opportunities are not yet rewrites; they are proposals to be reviewed before any SQL is generated.

Proposal filtering. A second LLM call acts as an adversarial critic over the proposed opportunities. The separation is intentional: a single LLM asked simultaneously for proposals and for skepticism produces neither, with proposal mode dominating. A separate pass instructed to assume the proposer is overconfident behaves differently. Filtering at this point also saves cost, since the refactoring stage that follows is the most token-expensive phase in the pipeline.

The critic looks for both slowdowns and correctness traps. The slowdown case rejects, for instance, proposals that look like reuse-driven savings but add a new write and scan without reducing downstream work. A correctness trap is illustrated in Figure 7: a parent table joins to a multi-tag child to compute per-item flags, and the proposal eliminates the join by reading the tag from a sibling that has been pre-deduplicated to one row per item. Doing so silently drops the secondary tags that the original join admitted. The critic returns one of three verdicts: *approve*, *reject*, or *conditional*. A conditional verdict means the proposal is net-positive only under a specific data property; that property is recorded as a constraint that the refactoring stage must respect.

Original: join admits all tags

```sql
SELECT i.item_id,
  MAX(CASE WHEN t.tag='A'
    THEN 1 ELSE 0 END) AS flag_a
FROM {{ ref('items') }} i
JOIN {{ ref('item_tags') }} t
  USING (item_id)
GROUP BY i.item_id
```

Proposal: drop join, read tag

```sql
SELECT s.item_id,
  CASE WHEN s.tag='A'
    THEN 1 ELSE 0 END AS flag_a
FROM {{ ref('items_staged') }} s
-- items_staged keeps one tag/item;
-- secondary tags are silently lost
```

Figure 7: Correctness trap rejected by the proposal critic. The original aggregates over all tags per item; the proposal reads from a sibling that has been deduplicated to one tag per item, and silently drops the rest.

Refactoring. The opportunities that survive filtering are passed, together with the analysis and the SQL of the affected nodes, to a third LLM call that produces concrete refactored SQL. The LLM

may rewrite existing nodes, mark nodes as removed, or introduce new shared nodes. Conditional opportunities from the previous stage carry their constraints into this prompt. Nodes with consumers outside the subgraph may have their internal SQL rewritten but must produce identical output rows, since their contract with downstream consumers is fixed.

Equivalence verification. A rewrite that survives filtering may still silently change the data. A bug rejected at the proposal stage can re-emerge inside a more complex rewrite that bundles several opportunities together: the same staged-input shortcut from Figure 7, for instance, can reappear nested inside a consolidated aggregation that the proposal critic approved on other grounds. DAGSmith treats equivalence as an empirical question against the pipeline's real data. A fourth LLM call, the *correctness critic*, examines the original-versus-refactored SQL of every rewritten node whose output is externally visible and, for each adopted opportunity, articulates the data invariant that must hold for the rewrite to be semantically safe. Each invariant is rendered as a SQL diagnostic, written so that it returns zero rows when the invariant holds and rows when it is violated. The diagnostics are executed against the target warehouse with a per-query byte cap. A non-empty result is a counterexample drawn from real data: the rewrite changes output. Counterexamples re-enter the pipeline as feedback. The failed invariants are appended to the refactoring prompt, and the LLM is asked to either propose a distinctly different strategy that respects them or restore the original logic. The loop continues until the refactor passes all diagnostics, the LLM declares no further distinct strategies, or a per-group attempt budget is exhausted, in which case the group is reverted to its original SQL and contributes no candidate.

Model families and propagation. A single subgraph can contain many near-duplicate models that perform the same operation over different data subsets. The subgraph analysis identifies such groups, designates a single *representative* per group, and records how the other members differ. Only the representative is taken through the four-stage sequence above; once it passes verification, the change is applied to the remaining members by a separate, lighter LLM call per member, executed in parallel. This is a scalability mechanism specific to projects whose models exhibit family structure; in projects without it, the family map collapses to singletons and the main sequence runs over every model directly.

3.3 Materialization Tuning

A candidate refactoring does not have a fixed cost: whether it improves the DAG depends on which of its nodes are materialized as tables and which are inlined as views, and the right answer differs from one candidate to the next. DAGSmith therefore evaluates every candidate under its own optimal materialization assignment, computed by the procedure described in this subsection. The original DAG, with no refactoring applied, is itself one of these candidates and serves as the baseline against which the others are scored.

Cost of a DAG configuration. Given a materialization assignment, the cost of the pipeline is computed by a single topological pass over the DAG. A table incurs its own predicted slot time once per execution and exposes its predicted cardinality to its children; a view incurs no slot time of its own and instead pushes its SQL features and its parents' cardinality forward into every node that consumes it, scaled by the number of times it is referenced. The features a node is scored on are therefore not the raw counts extracted from its own SQL alone but a vector that has accumulated the contributions of every view in its upstream chain. The cost a node pays consequently depends on the materialization of each of its ancestors, and flipping a single node from table to view can change the predicted cost of every descendant.

Cost model. Two Huber regression models, trained on the history of instrumented runs of the original pipeline, predict the per-node quantities consumed by the topological pass. The first predicts the cardinality of each materialized table from a set of SQL features extracted by parsing each node's compiled query: counts of joins, equi- and non-equi join keys, predicates broken down by kind (equality, range, IN), group-by clauses and keys, distinct operations, unions, and table scans. The second predicts slot time from a closely related set of features that adds window functions, total aggregations, and order-by clauses. Both predictions are conditioned on the cardinality of the node's parents under the current configuration. Both training and prediction operate on view-expanded feature vectors, so the regression sees, for each node, exactly the work it would actually perform under the configuration in question. Huber loss is used in place of squared error to limit the influence of a small number of very large queries.

Iterative local linearization. The total cost is a discrete non-linear function of the materialization vector $t \in \{0,1\}^{|V|}$ with interactions that propagate through every view chain in the DAG; exact minimization is intractable for pipelines of realistic size. DAGSmith therefore applies iterative local linearization, a standard successive-approximation scheme for discrete non-linearities: linearize the cost around the current configuration, solve the linear surrogate, move, and repeat until the configuration stops changing. The procedure converges to a locally consistent assignment rather than a global optimum, which is the practical price of feasibility.

Concretely, let $t_i \in \{0,1\}$ be the materialization of node i (table if $t_i = 1$, view if $t_i = 0$). At iteration k, with the current assignment $t^{(k)}$ fixed, DAGSmith measures two local quantities using the cost model: the baseline build cost $B_j^{(k)}$ of every node j under $t^{(k)}$ with its direct parents forced to tables, and the edge-local delta $U_{ij}^{(k)}$, the additional cost incurred at j when only parent i is flipped to a view (other parents held as tables). Treating $B^{(k)}$ and $U^{(k)}$ as constants for the duration of the iteration, the system solves a small ILP for the next assignment $t^{(k+1)}$:

$$\min_{t,\,y} \sum_{j \in V} t_j \, B_j^{(k)} \;+\; \sum_{i \to j \in E} y_{ij} \, U_{ij}^{(k)},$$

$$\text{s.t.} \quad y_{ij} \le 1 - t_i, \;\; y_{ij} \le t_j, \;\; y_{ij} \ge t_j - t_i, \;\; y_{ij} \ge 0,$$

where the auxiliary $y_{ij} \in [0,1]$ is constrained to take value $(1-t_i) \wedge t_j$: it is 1 precisely when j executes and reads from a parent i that has been inlined as a view, and 0 otherwise. A node j thus pays its baseline $B_j^{(k)}$ when materialized as a table and an additional $U_{ij}^{(k)}$ for each parent that is a view. To keep each step inside the region where the local coefficients remain valid, the ILP is constrained by a trust-region cap on the number of flips per iteration and a small flip penalty that damps oscillation. The new assignment becomes the current one and the loop repeats; B and U are recomputed

because flipping a node changes the predicted cardinalities seen by its descendants. The loop terminates on convergence, on revisiting a previously seen assignment, or after a fixed iteration cap. Two pieces of domain knowledge keep the search well behaved: nodes with many descendants are pinned as tables, since inlining them would cause their work to be redundantly recomputed for every consumer, and seed and source tables are fixed as tables throughout, since their materialization is not under the project's control.

From a candidate to a delta. For the original project, the procedure above runs directly: every model has a measured cardinality and slot time from the instrumented run, and the loop searches the materialization assignment that minimizes total cost. For a refactored candidate, models that did not exist in the original have no measurements. The candidate's compiled manifest is fed through the same topological pass, with the cost model invoked once per new model to predict its cardinality from its SQL features and the (predicted or measured) cardinality of its parents; from that point the candidate is indistinguishable from the original as far as the optimizer is concerned, and the same iterative loop produces its optimal materialization. The cost of the candidate under that materialization, subtracted from the cost of the original under its own, is the candidate's *delta*: the predicted cost reduction the refactoring delivers when adopted in isolation. The next subsection composes the candidates given these deltas.

3.4 Global Selection

The previous stage produces, for each candidate refactoring, a delta computed as if that candidate were adopted alone. Composing the candidates into a single optimized project is not a matter of taking them all: the structural analyses of Section 3.1 surface overlapping regions of the DAG, so the same model can belong to more than one group, and the LLM has rewritten each group in isolation; applying two refactorings that touch the same model would require a merge that no stage of the pipeline has produced and whose correctness no stage has verified. A separate constraint comes from *within* each group, where the LLM may have proposed several variants of the same refactoring opportunity, of which at most one can be adopted. Global selection picks the subset of candidates that maximizes total delta subject to both restrictions.

Conflict graph. Two groups are in conflict when they touch (rewrite, create, or remove) any of the same models. DAGSmith computes a conflict graph by intersecting the changed-model sets of every pair of groups; an edge is added whenever the intersection is non-empty. The graph is typically sparse, since the structural analyses tend to surface regions whose dataflow patterns differ, but it is rarely empty: shared upstream models that participate in multiple opportunities are the common source of edges.

ILP selection. Let $x_c \in \{0,1\}$ indicate whether candidate c is adopted, with delta δ_c, and let $z_g \in \{0,1\}$ indicate whether any variant from group g is adopted. DAGSmith solves

$$\max_{x,z} \sum_c \delta_c x_c \quad \text{s.t.} \quad \sum_{c \in g} x_c \le z_g \;\; \forall g, \quad z_g + z_{g'} \le 1 \;\; \forall (g,g') \in C,$$

where C is the edge set of the conflict graph. The first constraint enforces that at most one variant per group is adopted; the second enforces that no two conflicting groups are adopted together. The problem is small (one binary per candidate and per group, with a sparse constraint matrix) and is solved exactly by an off-the-shelf MILP solver. Once the subset is fixed, the materialization tuning of Section 3.3 is run once more on the combined project, since each candidate's delta was measured under its own isolated materialization assignment and the optimal assignment for their combination need not coincide with any one of them.

3.5 Frequency-Aware Optimization

The previous stages treated every node's slot time as a fixed per-execution cost. In a recurring SQL pipeline DAG this is an oversimplification: different sources update at different rates, different downstream consumers require fresh output at different rates, and the cost a node contributes to the pipeline is the cost of one execution multiplied by how often it actually executes. DAGSmith therefore augments the pipeline with a frequency dimension that enters at three layers. Frequencies are taken from the declared schedule at sink nodes and from observed update rates at source tables, and propagated through the interior of the DAG from these boundaries. Layers 1 and 2 reweight and reschedule the existing DAG without changing its structure; layer 3 contributes a new class of structural candidates back to the candidate generator of Section 3.1.

Layer 1: Frequency as cost weight. The materialization tuning of Section 3.3 and the selection of Section 3.4 both minimize total cost as if every model executed at the same rate. Replacing per-execution slot time with frequency-weighted slot time changes the answer: a refactoring that saves work on a daily-refresh model is worth more than the same per-execution savings on a weekly one. Mechanically, the local coefficients of the iterative linearization are reweighted before the ILP runs:

$$B_j^{(k)} \leftarrow f_j \cdot B_j^{(k)}, \qquad U_{ij}^{(k)} \leftarrow f_j \cdot U_{ij}^{(k)},$$

where f_j is node j's effective frequency. The rest of Section 3.3 is unchanged, and the selection ILP inherits the same reweighting through the deltas it consumes.

Layer 2: Reconciling scheduled and effective frequency. A model's effective frequency is not the rate at which its schedule fires; it is the rate at which its schedule fires *and* produces new output. Each model has a *demand* frequency f_j^{demand}, set by the rate at which its downstream consumers need fresh output, and a *data-changing* frequency f_j^{data}, set by the rate at which any of its upstream inputs actually change. The rate at which the model contributes useful work is

$$f_j = \min\left(f_j^{\text{demand}}, f_j^{\text{data}}\right).$$

Where the scheduled rate exceeds f_j, the model is being over-scheduled: it executes, reads its inputs, and produces output identical to what it produced last time. DAGSmith flags such models and reduces their schedule to f_j. This layer changes no SQL and no materialization, only the schedule, but the saving is immediate: every redundant execution is eliminated outright. The same f_j is what layer 1 uses as the cost weight, so the two layers compose.

Layer 3: Frequency-driven splitting hints. The first two layers leave the DAG structure intact. The third generates new structural candidates of a kind that the analyses of Section 3.1 cannot identify

on dataflow shape alone. When a model has parents with different data-changing frequencies, it inherits the fastest one; any work inside the model that depends only on slow-changing parents is therefore re-executed at the fast rate even though its inputs have not changed. DAGSmith identifies models whose parent frequencies are heterogeneous and whose slow-side computation is non-trivial, and emits the location to the candidate generator as a hint, alongside the reuse and prune analyses of Section 3.1. The hint enters the LLM stage in the same form as any other candidate region; the LLM proposes a frequency-split refactoring of the kind illustrated in motivating Example 3, in which the slow-side computation is extracted into a separate model that runs on the slow cadence and the original model becomes a lightweight join over its outputs. From there the candidate flows through the same filtering, refactoring, verification, scoring, and selection stages as any other, with layers 1 and 2 active throughout.

4 EVALUATION

We evaluate DAGSmith on real dbt projects, organized around three questions:

(1) How does DAGSmith compare to existing baselines for SQL pipeline optimization, and what classes of optimization account for the gap?
(2) How much does each component of DAGSmith's architecture contribute to the overall gain? We ablate the three optimization dimensions (refactoring, materialization, and frequency) and the dataflow-driven targeting that feeds the LLM stage.
(3) Is the underlying machinery sound and affordable? We measure the accuracy of the cost model that drives materialization decisions, the effectiveness of the equivalence-verification loop at catching unsafe rewrites, and the LLM cost of running the system end-to-end.

Summary of results. [PLACEHOLDER] will come back later. Note that we should explicitly say the cost is the cost on bigquery, not the overall cost

4.1 Experimental Setup

Workloads. We evaluate on two real dbt projects. *Tuva* is an open-source dbt package for healthcare data transformation. The full project comprises 842 SQL models organized across 17 modules. We use the claims preprocessing module, which contains 255 models (about 30% of the project) and is the subset whose models exhibit substantial computational complexity. We run it on a synthetic patient population of 10K, with the largest source table at 505 MB. *Stripe* is a payments-analytics project of 65 models, already factored by hand into intermediate stages so that the easy syntactic wins have been taken. We run it with the largest source table at 199 MB. The two projects together cover the range we want to evaluate: a large project that has accumulated cross-model patterns from multiple analysts, and a smaller, already-factored project where remaining gains must come from materialization, frequency, or semantic restructuring beyond syntactic deduplication.

Testbed. Experiments run on BigQuery with a 100-slot reservation on the Standard edition under capacity-based pricing. We report slot-time as the primary cost metric, since slot-time is the billing unit under capacity pricing, and elapsed time as a secondary latency metric. Each reported number is the median of three runs. To simulate realistic production schedules, we use three frequency profiles per project that assign root and leaf models to hourly, daily, or weekly cadences, with interior models derived by propagation. Table 2 summarizes the average cadence mix per project; all frequency-weighted results in this section are averaged across the three profiles.

Table 2: Frequency profile averages per project. Each row is the fraction of root and leaf models at each cadence, averaged across three profiles.

	Roots			Leaves		
Project	wk	day	hr	wk	day	hr
Tuva	0.18	0.57	0.25	0.07	0.20	0.73
Stripe	0.13	0.60	0.27	0.05	0.52	0.43

Cost metrics. We report two views of cost. *Single-run cost* is the cost of executing the full DAG once. *Frequency-weighted cost* weights each model's per-execution cost by the rate at which it actually needs to run, summed over the DAG; under capacity pricing this corresponds to the project's billed cost over a period. Refactoring and materialization affect both metrics; frequency optimization affects only the frequency-weighted metric. Improvements are reported as percent reduction relative to the original project.

Baselines. We compare against three baselines spanning the design space of SQL pipeline optimization.

LearnedRewrite [**?**] is a rule-based per-query rewriter. It applies a portfolio of equivalence-preserving transformations to each model's compiled SQL independently, guided by a learned policy that selects which rules to apply and in what order. It is competitive on standalone analytical queries but has no view of the surrounding DAG.

GenRewrite [13] is an LLM-based per-query rewriter. It introduces natural-language rewrite rules that transfer knowledge across queries and a counterexample-guided loop that iteratively corrects syntactic and semantic errors in the rewrites. Like LearnedRewrite, it operates one model at a time.

CSE is a cross-model common subexpression exploitation baseline patterned after Silva-Larson-Zhou [**?**], which detects subexpressions shared across a SCOPE script's stages and materializes each once, re-optimized at the query optimizer's memo. The memo step is not portable, since managed warehouses do not expose their optimizer's internal state to external tools. We adapt the approach by detecting shared subexpressions at the AST level and selecting which to materialize heuristically. CSE is the strongest available position for a syntactic, cross-model approach on a managed warehouse.

4.2 Comparison with Baselines

Table 3 reports the improvement of each method over the original project on Tuva and Stripe. DAGSmith leads on every metric on both projects. The single-run slot column captures the cost of one DAG

Table 3: Improvement over the original project (% reduction; higher is better). Medians of three runs on a 100-slot BigQuery reservation.

	Single-run		Freq-weighted	
	Elapsed	Slot	Elapsed	Slot
Tuva (255 models)				
LearnedRewrite	−19.6	−2.5	−20.2	−3.0
GenRewrite	9.7	17.7	21.5	15.1
CSE	12.2	34.6	27.7	37.9
DAGSmith	**30.9**	**51.1**	**42.6**	**67.7**
Stripe (65 models)				
LearnedRewrite	2.6	2.7	−0.5	4.5
GenRewrite	7.6	7.1	1.1	9.3
CSE	−3.2	−5.6	−3.1	−0.6
DAGSmith	**40.0**	**47.4**	**36.1**	**54.8**

execution and is affected only by refactoring and materialization. The frequency-weighted slot column weights each model's per-execution cost by the rate at which the model actually needs to run, summed over the DAG; under capacity pricing this corresponds to the project's billed cost over a period. The gap between the two columns is the contribution of frequency optimization, which affects only the frequency-weighted view.

Each baseline falls short for a different structural reason. We name the limit in each case, then summarize the classes of optimization that account for the gap above the strongest baseline.

LearnedRewrite: rules tuned for standalone queries do not generalize to dbt models. LearnedRewrite applies equivalence-preserving rewrite rules independently to each model. On Tuva, the one rule that fires at scale converts `SELECT DISTINCT . . . JOIN` into a join of two pre-deduplicated inputs. It wins on models whose join keys are nearly unique (the pre-dedup is cheap and shrinks the join) and loses on models whose join keys are coarse (the pre-dedup forces a full grouping pass of a wide table and the resulting plan is worse than what the warehouse planner would have chosen). The wins and losses cancel. On Stripe, an already-factored project, the syntactic shape that triggers the rule is largely absent and the rule barely fires. The headlines on both projects are essentially flat. Rule-based per-query rewriting has nothing left to do once a planner has handled the easy cases and cross-model structure is out of reach.

GenRewrite: per-model LLM reach helps where there is slack, hurts where there is not. GenRewrite finds real per-model algorithmic rewrites that a rule-based system cannot: dynamic predicate derivation through inequality joins, functional-dependency-aware splitting of wide enrichment joins, replacement of $O(n^2)$ self-joins with $O(n)$ window-function plans. On Tuva these wins account for most of its 17.7% slot reduction. On Stripe the same approach gains far less (+7.1% slot, against +17.7% on Tuva), and a single-run reading suggests it can be net negative on a project this small. The LLM rewrites the project's largest models with column projections the planner was already applying for free, and disrupts plans the warehouse had already optimized; it has no feedback signal telling it that nothing needs to change. The lesson generalizes: an LLM rewriter operating one model at a time helps where there is algorithmic slack a planner cannot resolve and hurts where there is none.

CSE: syntactic matching sees duplication, not equivalence. CSE detects subexpressions shared across models at the AST level and is the only baseline with cross-model scope. It is strong where the same SQL appears in multiple places, collapsing Tuva's largest classifier family from 6,135 s to 613 s on syntactic match alone, and weak everywhere else. Models that compute the same operation over different inputs, with different join orders, filter values, and upstream references, share the work in substance but not in SQL; CSE cannot match across them. On Stripe, where a human engineer has already taken the easy syntactic wins, CSE has almost nothing to do.

DAGSmith: four patterns that the baselines structurally cannot reach. The classes of optimization that produce DAGSmith's gap above the strongest baseline fall into four recurring patterns. The first two are forms of cross-model refactoring that go beyond syntactic matching; the next two exercise dimensions outside compiled SQL altogether. Each is illustrated with one concrete instance from the workloads.

(1) Semantic-equivalence factorization across dissimilar siblings. Models that compute the same logical operation over different inputs are folded into one shared intermediate, even when their SQL has no syntactic overlap. On Tuva, six inpatient encounter-rollup models compute the same per-encounter aggregations over different encounter types; DAGSmith synthesizes a single rollup that carries the encounter type as a column and replaces all six bodies with lookups against it. The family drops from 914 s to 211 s, where CSE recovers only 127 s of that gap. This is the regime where syntactic AST matching cannot reach but semantic equivalence is clear once the LLM stage reads each model.

(2) Cross-model narrowing. When many consumers each scan a wide table to read a small subset of its columns, the projection is extracted into a narrow intermediate consumed by all of them. The pattern shows up wherever a fan-out exists: many readers, one wide source, narrow per-reader needs. On Tuva, a 14-branch union consumer reads a single column from a wide claim-line table on each branch; once DAGSmith extracts the narrow projection, the consumer drops from 1,420 s to 374 s. The shared work here is column-level; per-query rewriters do not see it and AST matching does not find it because each branch references a different upstream model.

(3) Materialization revision unlocks planner reach. When a model is materialized as a table, the warehouse planner sees an opaque scan; when it is materialized as a view, the planner sees the full source SQL and can push predicates, prune columns, and reorder joins across the staging boundary. DAGSmith treats materialization as an optimization variable and revises it across the project. On Stripe, flipping the staging layer of a wide reporting model from tables to views drops the model's bytes scanned by a third and its slot by a factor of 2.5, on byte-identical model SQL. The downstream savings are produced entirely by the planner, but they exist only because DAGSmith changed the materialization choice; no approach that operates on compiled SQL can reach them.

(4) Materialization revision tied to refactored structure. The right materialization for the original project is not the right materialization for the refactored project. Once a refactoring changes which models read from which, the build cost of an upstream table can stop being worth paying. On Tuva, six voting and normalize stages were materialized as tables in the original project, each feeding a single downstream consumer; once DAGSmith refactors the consumers, the tuner flips the upstream stages to views, and the family's slot drops by 24%. CSE, which applies the original project's materialization choices to its rewrites, regresses on the same family. This is direct evidence that refactoring and materialization must be tuned jointly: neither pass in isolation finds the configuration that the joint pass does.

Frequency optimization is the fifth dimension and contributes the gap between the single-run and frequency-weighted columns of Table 3: DAGSmith reduces the firing rate of models whose inputs change less often than they execute, eliminating redundant runs outright. Like materialization, it is invisible to any approach that operates on compiled SQL.

Tuva and Stripe exercise these patterns in different proportions. Tuva's gap above the strongest baseline is dominated by patterns 1 and 2 (cross-model refactoring); Stripe's is dominated by pattern 3 (materialization revision). The same pipeline produces both outcomes because the optimizer chooses what to apply to the project in front of it, rather than committing to a single mechanism upfront. The next subsection quantifies the contributions component by component.

4.3 Component Ablation

Section 4.2 described the *classes of optimization* responsible for DAGSmith's gap above the baselines. This subsection asks a different question: among DAGSmith's own architectural components, how much does each contribute, and how do they interact? We ablate along two axes: the three optimization dimensions of refactoring (R), materialization tuning (M), and frequency optimization (F); and the dataflow analysis that targets the LLM stage. The ablation is performed on Tuva alone, because Tuva is the larger and more structurally varied of the two projects and exercises all three dimensions meaningfully; Stripe is small enough that its single-run signal sits close to the run-to-run noise band, making fine-grained dimension comparisons unreliable.

4.3.1 Three Optimization Dimensions. Table 4 reports the improvement of each subset of $\{R, M, F\}$ over the original Tuva project.

The numerics admit four observations, each one reflecting a property of how the dimensions are designed to interact.

Slot reductions exceed elapsed reductions across every variant. Slot measures the total CPU-time the project consumes; elapsed measures the wall-clock time from first model to last. DAGSmith removes work parallel to the critical path more aggressively than work on the critical path itself: a refactoring that consolidates many independent sibling models into one shared intermediate saves slot proportional to the number of siblings but saves elapsed only the slowest sibling's share. The slot-vs-elapsed gap is largest for R-only (49.1% vs 40.0%), where this effect is most direct, and remains visible across the rest of the table.

Table 4: Tuva: dimension ablation (% reduction over the original project, averaged across the three Tuva profiles in Table 2). R = refactoring, M = materialization tuning, F = frequency optimization.

Variant	Elapsed	Slot
original	–	–
F only	13.2	11.0
M only	34.7	37.0
R only	40.0	49.1
M + F	31.0	54.0
R + M	43.1	54.7
R + M + F (full)	**42.6**	**67.7**

R-only is closer to R+M than the headline suggests, because R is not pure refactoring. When DAGSmith's refactoring stage introduces a new model, it also assigns that model an optimal materialization as part of producing the candidate (§**??**). The R-only variant therefore already tunes the materialization of newly created models; what M adds on top is project-wide retuning of pre-existing models, which is a smaller marginal change. The gap between R-only (49.1%) and R+M (54.7%) is correspondingly modest. M-only (37.0%) is lower than R-only because it cannot create models at all, so it can only revise materialization on the original DAG.

Refactoring and materialization are coupled. R+M (54.7%) exceeds the larger of the two singletons (R: 49.1%) by 5.6 points. The refactored DAG admits a different optimal materialization than the original DAG, and the joint pass finds savings that neither stage produces in isolation. The Section 4.2 argument that refactoring and materialization decisions must be made jointly is made concrete here: pattern 4 of Section 4.2 contributes to R+M but not to R or M alone.

Frequency's increment depends on what came before it. F alone delivers 11.0%; F adds 17.0 points on top of M (54.0 vs 37.0); F adds 13.0 points on top of R+M (67.7 vs 54.7). F's reach is bounded by the over-scheduled work present in the project, and that pool itself depends on the prior dimensions. Refactoring can change which models are over-scheduled (a refactoring that splits a fast-cadence model into a slow rollup and a fast lookup moves work onto the appropriate cadence and removes it from F's surface), while materialization tuning leaves the over-schedule pool largely intact. F therefore has more to remove on top of M than on top of R+M. The non-collapsing increment confirms that F is structurally orthogonal to R and M and not a redundant pass; the size of the increment confirms that the three dimensions interact in non-trivial ways.

4.3.2 Dataflow Analysis. The dimension ablation above takes DAGSmith's candidate set as given. This experiment instead isolates the contribution of the dataflow analysis that produces that set (§3.1): does structural targeting direct the LLM stage to more productive regions of the DAG than alternatives that do not use it? The central design principle is that the LLM-call budget must be held fixed across all arms. Otherwise the comparison would merely reflect that more LLM invocations uncover more refactorings, rather than

whether dataflow analysis spends a fixed budget more effectively. In the standard pipeline, dataflow analysis selects at most `[20]` subgraphs on Tuva, each capped at `[60]` nodes, and the LLM is given `[3]` diagnostic-and-correction attempts per subgraph. We hold this per-subgraph attempt budget fixed across every arm and compare against two alternatives.

Whole-DAG refactoring. The first alternative supplies the entire project to the LLM under the same 3-attempt budget. This arm tests whether restricting the LLM's attention to a bounded region is necessary at all. On Tuva, the whole DAG fits within the context window (134k input tokens), and the analysis phase succeeds: the LLM identifies most of the opportunities that our pipeline surfaces. Implementation then fails for three compounding reasons. First, length: the response must restate every rewritten model in full, growing long enough that the LLM fails to finish and silently truncates, leaving dangling references. The truncated output is not a runnable project, so this arm yields no valid DAG to measure; we therefore report its behavior rather than a cost. Second, interdependence: the more models a single attempt rewrites, the more cross-model references it must keep consistent, and a fix to one model breaks an assumption in another. Third, non-convergence: because every broad attempt breaks somewhere, the correction loop retreats in scope, with the three attempts rewriting successively fewer models, 51, then 35, then 8. This exposes a dilemma whole-DAG refactoring cannot escape: a broad rewrite captures most gain but touches too many interdependent models to verify, while a rewrite narrow enough to verify captures only a fraction of the gain. With no way to decompose the project, the LLM retreats to the safe, low-gain end. Dataflow analysis resolves the dilemma: each subgraph is small enough to verify, yet the pipeline still reaches the high-value regions by examining them one at a time.

Random subgraphs. The second alternative keeps everything fixed except the selection rule: the same subgraph count, the same size cap, and the same downstream refactoring procedure, with dataflow-guided selection replaced by random selection. We draw a model uniformly at random and take its parents, children, and siblings as a subgraph, repeating until the count matches the standard pipeline and discarding any neighborhood over the node cap. This isolates the contribution of *where* the pipeline chooses to look. Random selection reaches a single-run slot reduction of 43.1% and a frequency-weighted reduction of 58.0%, against 51.1% and 67.7% for dataflow-guided selection. The gap reflects a mismatch between two structures. An edge-based subgraph groups a model with its dependency neighbors, but the models worth refactoring together are not its dependency neighbors: they are models elsewhere in the project that compute the same pattern. Two sibling pipelines may each apply the same expensive transformation to different inputs without sharing a single edge, so no edge-based neighborhood contains both, and widening the neighborhood with more hops only adds dependency-adjacent models that are irrelevant to the refactoring. Dataflow-guided selection groups on the structure of the computation rather than on dependency edges, so it places the models that compute the same pattern in one subgraph regardless of where they sit in the DAG.

4.4 System Characterization

This subsection asks whether the machinery that produces the gains above is itself sound and affordable. We measure the accuracy of the cost model that drives materialization and refactoring decisions, the effectiveness of the equivalence-verification loop at catching unsafe rewrites, and the LLM cost of running DAGSmith end-to-end.

4.4.1 Cost Model Accuracy. [PLACEHOLDER] To be filled in. Reports the accuracy of the Huber-regression cardinality and slot-time predictors against measured values on the original and refactored projects.

4.4.2 Equivalence-Verification Effectiveness. [PLACEHOLDER] Reports how the equivalence-verification loop behaves on the refactoring candidates produced for Tuva and Stripe: the number of candidates that pass on the first attempt, the number that require one or more revision rounds, and the number rejected outright. For candidates the loop rejects or revises, we characterize the kinds of unsafe transformations caught (e.g., the secondary-tag drop illustrated in Figure **??**). The end-to-end question is whether the loop is conservative enough to be trusted as the project's safety net, while not so conservative that real refactorings are discarded.

4.4.3 LLM Cost. [PLACEHOLDER] Reports total LLM tokens consumed and dollar cost of running DAGSmith end-to-end on Tuva and on Stripe, broken down by stage (subgraph analysis and opportunity proposal, proposal filtering, refactoring, equivalence verification, family propagation). Numbers TBD.

5 RELATED WORK

The most closely related lines of work are database query optimization, source-to-source SQL rewriting, multi-query optimization, materialized-view recommendation, workflow scheduling, and data lineage or clone analysis for analytics pipelines. DAGSmith differs from these lines by treating the explicit SQL pipeline DAG itself as the optimization object, allowing it to restructure node boundaries and reason jointly about semantics, materialization, and refresh frequency.

Database query optimizers remain essential but orthogonal: they choose physical plans for one submitted SQL statement. DAGSmith operates before that point, changing the maintained SQL pipeline program that produces those statements. Source-to-source SQL rewriters such as *SlabCity* [10], *GenRewrite* [13], *WeTune* [21], and *R-Bot* [18] also operate before the database optimizer, but they optimize one statement at a time and do not change the dependency graph around it.

Multi-query optimization and common-subexpression sharing reason across multiple statements, but typically over flat workloads and mostly through syntactic overlap. They do not usually exploit output contracts, declared refresh cadences, materialization choices, or stable developer-maintained node boundaries. Materialized-view selection optimizes persistence decisions, but does not usually rewrite the graph to move computation across freshness boundaries or remove intermediate contracts. Workflow systems such as Airflow [2], Dagster [4], Prefect [15], Argo [3], Luigi [14], SQLMesh [19], and Databricks Workflows [5] expose

dependencies and schedules, but their optimizers primarily decide ordering, invalidation, or execution policy rather than semantic SQL rewrites. DAGSmith targets the gap between these areas: dependency-aware source-to-source optimization for recurring SQL pipeline DAGs.

6 CONCLUSION

Explicit SQL pipeline DAGs create an optimization setting that is richer than traditional one-query-at-a-time rewriting. Once dependencies, node contracts, materialization choices, and refresh behavior are visible, the optimizer can restructure the pipeline itself rather than merely improve individual statements. DAGSmith is a step toward that goal: it treats dependency-aware pipeline optimization as a joint problem over graph structure, materialization, and frequency, and combines structural analysis with LLM-guided refactoring and verification to explore that space.

The core claim is that recurring SQL pipeline DAGs expose whole-pipeline optimization opportunities that are invisible to query-local rewriting, and exploiting them requires a system designed for dependency-aware reasoning.

REFERENCES

[1] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes in SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases*. 496–505.

[2] Apache Airflow. 2026. DAGs. https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html. Accessed 2026-05-26.

[3] Argo Workflows. 2026. DAG. https://argo-workflows.readthedocs.io/en/latest/walk-through/dag/. Accessed 2026-05-26.

[4] Dagster Labs. 2026. Software-defined assets. https://docs.dagster.io/. Accessed 2026-05-26.

[5] Databricks. 2026. Configure task dependencies. https://docs.databricks.com/aws/en/jobs/run-if. Accessed 2026-05-26.

[6] Databricks. 2026. What is the medallion lakehouse architecture? https://learn.microsoft.com/en-us/azure/databricks/lakehouse/medallion. Accessed 2026-05-26.

[7] dbt Labs. 2024. The next big step forwards for analytics engineering. https://www.getdbt.com/blog/analytics-engineering-next-step-forwards. Accessed 2026-05-26.

[8] dbt Labs. 2024. What is data lineage, and why do you need it? https://www.getdbt.com/blog/what-is-data-lineage. Accessed 2026-05-26.

[9] dbt Labs. 2025. 2025 State of Analytics Engineering Report. https://www.getdbt.com/resources/reports/state-of-analytics-engineering-2025. Accessed 2026-05-26.

[10] Rui Dong, Jie Liu, Yuxuan Zhu, Barzan Mozafari, Xinyu Wang, and Cong Yan. 2023. SlabCity: Whole-Query Optimization using Program Synthesis. https://par.nsf.gov/servlets/purl/10451715.

[11] Tristan Handy. 2026. What, exactly, is dbt? https://www.getdbt.com/blog/what-exactly-is-dbt. Accessed 2026-05-30.

[12] Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1996. Implementing Data Cubes Efficiently. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 205–216. https://doi.org/10.1145/233269.233333

[13] Jie Liu and Barzan Mozafari. 2024. Query Rewriting via Large Language Models. https://arxiv.org/abs/2403.09060. *CoRR* abs/2403.09060 (2024).

[14] Luigi Contributors. 2026. Building workflows. https://luigi.readthedocs.io/en/stable/workflows.html. Accessed 2026-05-26.

[15] Prefect Technologies. 2026. Tasks. https://docs.prefect.io/v3/concepts/tasks. Accessed 2026-05-26.

[16] Prasan Roy, S. Seshadri, S. Sudarshan, and Siddhesh Bhobe. 2000. Efficient and Extensible Algorithms for Multi Query Optimization. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 249–260. https://doi.org/10.1145/342009.335419

[17] Timos K. Sellis. 1988. Multiple-Query Optimization. *ACM Transactions on Database Systems* 13, 1 (1988), 23–52.

[18] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. 2024. R-Bot: An LLM-based Query Rewrite System. *CoRR* abs/2412.01661 (2024).

[19] Tobiko Data. 2026. SQLMesh Overview. https://sqlmesh.readthedocs.io/en/stable/concepts/overview/. Accessed 2026-05-26.

[20] Adrian Vogelsgesang, Michael Haubenschild, Jan Finis, Alfons Kemper, Viktor Leis, Tobias Muehlbauer, Thomas Neumann, and Manuel Then. 2018. Get Real: How Benchmarks Fail to Represent the Real World. In *Proceedings of the Workshop on Testing Database Systems (DBTest '18)*. Association for Computing Machinery, New York, NY, USA, Article 4, 6 pages. https://doi.org/10.1145/3209950.3209952

[21] Zhaoguo Wang, Zhou Zhou, Yicun Yang, Haoran Ding, Gansen Hu, Ding Ding, Chuzhe Tang, Haibo Chen, and Jinyang Li. 2022. WeTune: Automatic Discovery and Verification of Query Rewrite Rules. In *Proceedings of the 2022 International Conference on Management of Data*. 94–107. https://doi.org/10.1145/3514221.3526125